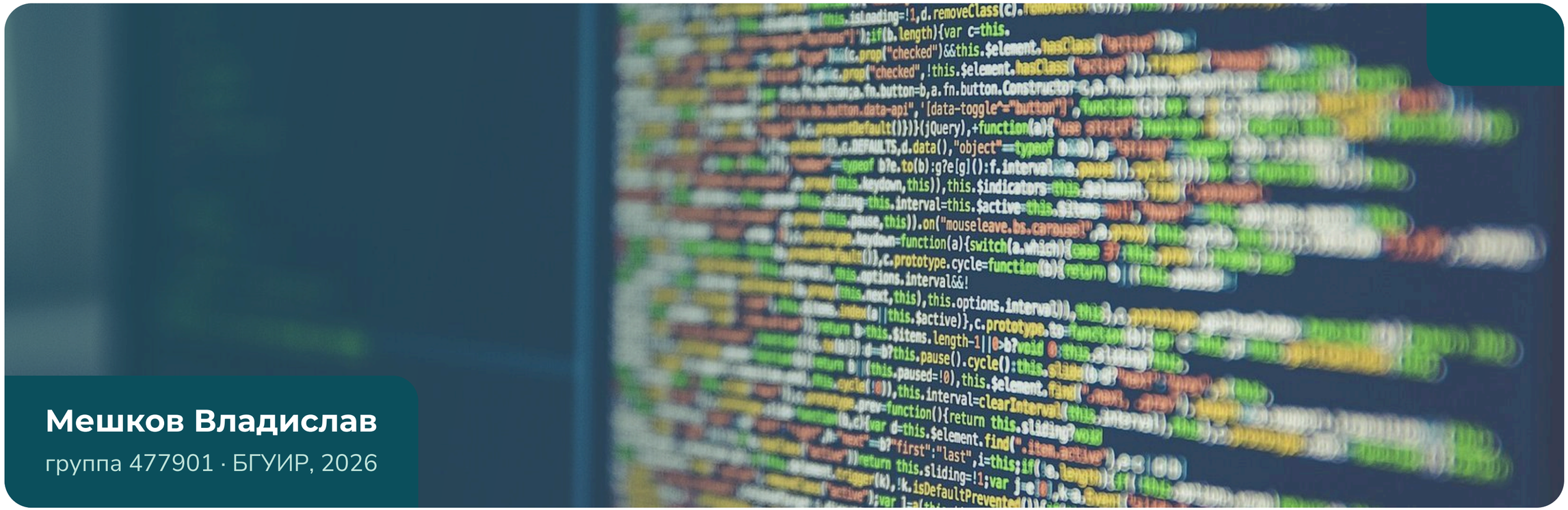
Маркетинг программных продуктов и услуг

HARNESS в ИИ

Что на самом деле покупает клиент, когда покупает ИИ-продукт



Мешков Владислав

группа 477901 · БГУИР, 2026

Одни и те же модели у всех

GPT
OpenAI

Claude
Anthropic

Gemini
Google

доступны любому разработчику через API

Проблема

Модели доступны всем. Продукты — нет.

GPT, Claude и Gemini подключает через API любой разработчик. А продукты на их основе работают очень по-разному.

30+ → 5

место в рейтинге TerminalBench 2.0: та же модель, те же веса, другая обвязка

76,4%

pass rate: LLM сам оптимизировал инфраструктуру и обошёл системы, собранные вручную

Источник: LangChain, по статье Akshay (перевод на Хабре)

Определение

Агент = Модель + Harness

Агент — поведение, которое видит пользователь. Harness — вся программная инфраструктура вокруг LLM: механизм, который это поведение производит.

agent_loop — схема

```
while not done:
    reply = llm(prompt)
    run(reply.tool_calls)
```

«Если ты не модель — ты харнесс»

Vivek Trivedy, LangChain

Три слоя вокруг модели

От промпта к harness

1 **Prompt engineering**
формирует инструкции, которые получает модель

2 **Context engineering**
управляет тем, что модель видит и когда

3 **Harness engineering**
оба слоя плюс инструменты, состояние, ошибки, проверка и безопасность

Аналогия Beren Millidge, 2023

Harness — это операционная система

Голая LLM — как процессор без памяти, диска и ввода-вывода



В компьютере: RAM

Контекстное окно

быстрое, но ограниченное

В компьютере: диск

Внешние базы данных

большие, но медленные

В компьютере: драйверы

Инструменты

руки агента: файлы, поиск, код

В компьютере: ОС

Harness

управляет всем остальным

«Мы заново изобрели архитектуру фон Неймана»

Компоненты production-харнесса

Из чего состоит продукт

ДЕЙСТВУЕТ	ПОМНИТ	НЕ ПОДВОДИТ	БЕЗОПАСЕН И РАСТЁТ
Доводит дело до конца	**Не теряет нить**	**Ловит свои ошибки**	**Знает свои границы**
• оркестрационный цикл • инструменты • парсинг вывода	• память • управление контекстом • управление состоянием • конструирование промпта	• обработка ошибок • циклы верификации	• ограничители и безопасность • оркестрация субагентов
АТРИБУТ 1	АТРИБУТ 2	АТРИБУТ 3	АТРИБУТ 4

МАРКЕТИНГ Это характеристики для карточки сравнения продуктов

Компонент 1 · оркестрационный цикл

Цикл — обычный while



1 Собрать промпт
система, инструменты, память, история, запрос

2 Вызвать модель
она отвечает текстом и/или просит вызвать инструмент

3 Выполнить инструмент
проверка прав и изоляция; ошибка — тоже результат

4 Добавить результат в контекст
и снова к шагу 1; вызовов нет — это финальный ответ

Цикл останавливается: ответ без вызовов, лимит ходов, бюджет токенов, tripwire, прерывание пользователем. Anthropic называет свой runtime «тупым циклом»: ум в модели, работа в harness.

МАРКЕТИНГ Для клиента это «агент сам довёл дело до конца»

Контекст — дефицитный ресурс

1 **Compaction**
сжатие истории у лимита: решения сохраняем, мусор выбрасываем

2 **Маскирование**
старые выводы инструментов скрыты, сами вызовы видны

30%+

на столько падает качество, когда важное оказывается в середине окна

Chroma; «Lost in the Middle», Stanford

3 **По требованию**
данные грузятся по ходу: grep и glob вместо целых файлов

4 **Субагенты**
исследуют широко, возвращают саммари на 1000–2000 токенов

Память в Claude Code

Индекс
~150 символов на запись, всегда в контексте

Файлы по темам
подгружаются по требованию

Транскрипты
доступны только через поиск

МАРКЕТИНГ Клиент ощущает это как «агент не тупеет к концу работы»

Ошибки, проверка, безопасность

Надёжность — то, за что платят

90,4%

Ошибки копятся

10 шагов по 99% успеха дают на выходе лишь ~90,4%. Сбой возвращают модели как результат, а число повторов ограничивают.

У Stripe — не больше двух

2–3×

Проверка работы

Тесты, скриншоты через Playwright или отдельный LLM-судья: умение проверять себя поднимает качество в 2–3 раза.

Boris Cherny, создатель Claude Code

~40

Права под контролем

Модель решает, что попытаться, а система — что разрешено. В Claude Code так контролируются около 40 возможностей.

Доверие → проверка вызова → подтверждение человеком

Компонент 11 · оркестрация субагентов

Как делегируют подзадачи

Claude Code

Fork
копия родительского контекста

Teammate
отдельная панель терминала, общение через файлы-mailbox

Worktree
отдельный git worktree: своя ветка на каждого агента

Agents SDK OpenAI

Agents-as-tools
специалист решает ограниченную подзадачу

Handoffs
специалист берёт на себя полное управление

LangGraph

Вложенные графы
субагенты — это вложенные графы состояний

Сначала один агент

Anthropic и OpenAI советуют максимально использовать одного. Делить стоит, когда пересекающихся инструментов больше десяти или домены задач явно разные: у мультиагентности есть накладные расходы на маршрутизацию и потери контекста при handoff.

Ralph Loop · Anthropic

Задача длиннее контекстного окна

Один раз

Initializer Agent

- init-скрипт
- progress-файл
- список фич
- первый git-коммит

Каждая следующая сессия

Coding Agent

1. читает git-логи и progress-файл, чтобы сориентироваться
2. берёт незавершённую фичу с наивысшим приоритетом
3. работает над ней
4. делает коммит и пишет саммари

Claude Code
git-коммиты как чекпоинты, progress-файлы как структурированные черновики

LangGraph
типизированное состояние, чекпоинты на границах super-step, возобновление и time-travel debugging

OpenAI
четыре стратегии: память приложения, SDK sessions, Conversations API или previous_response_id

МАРКЕТИНГ Клиенту важна непрерывность: работа не теряется между сессиями

Реализации на рынке

Один паттерн — разные обещания

Claude Code Anthropic	Тонкий harness: ставка на улучшение модели, лишние шаги убирают
Codex OpenAI	Один harness на все интерфейсы: CLI, VS Code и веб
LangGraph LangChain	Явный граф состояний: ставка на контроль над каждым шагом
CrewAI	Команда агентов: у каждого роль, цель и свои инструменты
AutoGen Microsoft	Агенты решают задачу в диалоге друг с другом

МАРКЕТИНГ Модель одна — продают подход и позиционирование

Проектирование

Семь решений архитектора

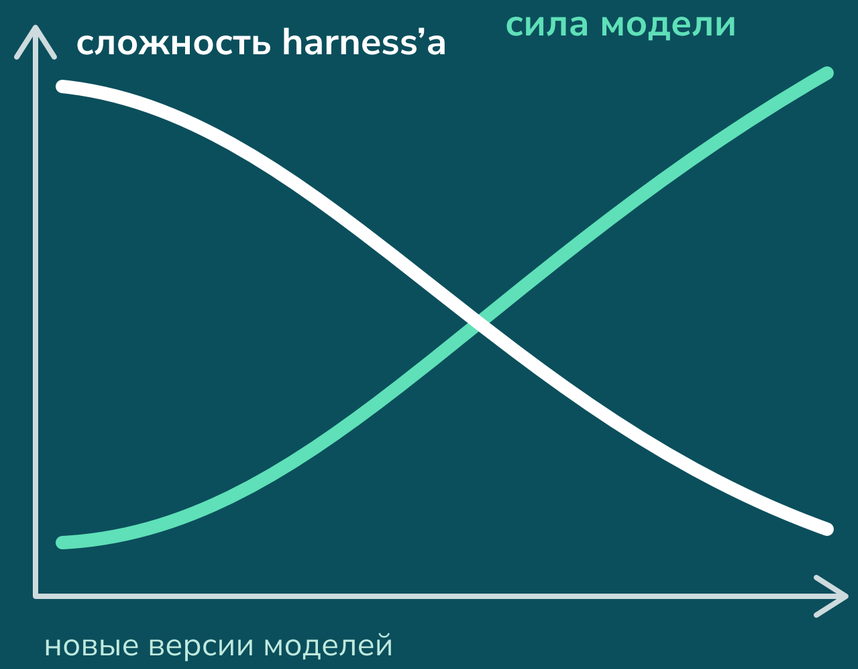
1	**Один агент или несколько**	Сначала один; делить при ~10 и более пересекающихся инструментах
2	**ReAct или plan-and-execute**	ReAct гибче, но дороже на шаг; LLMCompiler ускоряет **в 3,6 раза**
3	**Управление контекстом**	Пять подходов; ACON: на **26–54%** меньше токенов при точности 95%+
4	**Цикл проверки**	Тесты и линтеры — точно; LLM-судья ловит смысл, но добавляет задержку
5	**Права и безопасность**	Свободно — быстро, но рискованно; строго — безопасно, но медленно
6	**Набор инструментов**	Больше — часто хуже: Vercel убрал **80%** инструментов v0, и стало лучше
7	**Толщина harness'а**	Anthropic ставит на тонкий, графовые фреймворки — на явный контроль

Жизненный цикл harness’а

Леса убирают, когда здание готово

- Модели улучшаются — сложность harness’а должна **снижаться**
- **Manus** перестраивали пять раз за шесть месяцев, каждый раз убирая сложность
- Anthropic убирает шаги планирования, когда модель их **усвоила**
- Тест: модель сильнее, а harness не сложнее — значит, дизайн верный

Чем сильнее модель, тем тоньше harness



концептуальная схема, без числовых данных

Взгляд маркетолога

Модель — товар, harness — продукт

1 **Модели доступны всем**
подключить GPT, Claude или Gemini может любой разработчик

2 **Отличие уходит в обвязку**
на одной модели продукты работают по-разному

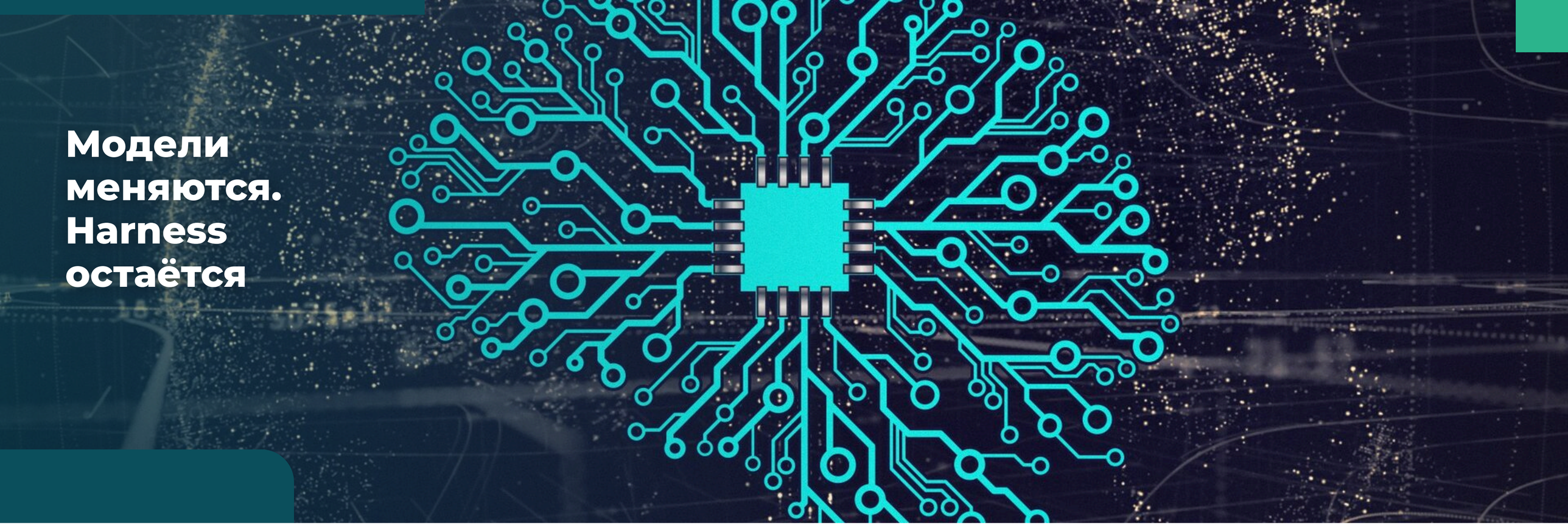
3 **Harness — преимущество**
его трудно скопировать: в нём опыт и ошибки команды

Чек-лист

4 вопроса любому ИИ-продукту

- ? Что он помнит между сессиями?
- ? Как проверяет свою работу?
- ? Что ему разрешено без человека?
- ? Что делает, когда ошибается?

Клиенту продают не «доступ к ИИ», а выполненную задачу



СПАСИБО ЗА ВНИМАНИЕ

Не вините модель. Смотрите на harness.

Мешков Владислав · группа 477901 · БГУИР, 2026 · по материалам статьи Akshay (перевод на Хабре)

Фото: Markus Spiske (CC0); mikemacmarketing, CC BY 2.0 · логотипы: Simple Icons (CC0), Wikimedia Commons; товарные знаки принадлежат владельцам